Towards General Purpose Robots

Manu Gaur

Long-Horizon Robotics is Memory-Bottlenecked

Without an abstraction of the past, it is impossible to form a causal understanding of how the world evolves and to model its dynamics.

- As the individual skills become more robust, the bottleneck is increasingly in how the robot deploys these skills to solve complex tasks, not the skills themselves.
- **Complex tasks need narrative memory**. "Making coffee" means tracking task stage, object locations (even out of view), and what's already been tried (to avoid repeating mistakes)
- **Attention alone can't scale.** Keeping a full observation history over minutes is infeasible, but discarding it leads to incoherent behavior.

The model needs to keep track of time to cook the grilled cheese

The model needs to remember its progress wiping the window

Why Future Prediction?

Predicting the future forces the model to learn:

- operate at a higher **temporal abstraction** – what task stage am I in, and what comes next?

- **Dynamics** – how objects move, interact, and change state

- a ***persistent memory*** to deal with **partial observability** – inferring what's out of view from what was seen before

This is the same principle the brain uses – prediction error drives attention, memory and learning. [Friston, 2010]

1. Karl Friston. The free-energy principle: a unified brain theory? Nature reviews neuroscience, 2010.

Problem Formulation

Future prediction can:
- operate at higher temporal abstraction
- encode task stage → better for multimodal futures
- deal with partial observability

LLM can do all the above; Q: how about spatial causality (MPC)?

Problems with training future predictor:
- mode collapse with MSE
  - generative or contrastive?
- compounding errors: curriculum over horizon lengths (can do with arch as well)
- shortcut learning

Long horizon BPTT not feasible:
GPU and Data constrained (RL has data, but unstable over long horizons)
- stage 1: linear TTT over chunks
- stage 2: Encoder-decoder autoregressive over long horizons.
- stage 3: downstream transfer (policy head and VQA head)

Pixels vs Latents?

Planning horizon
- low level policy: reactive, CroBo
- MPC/MBRL: forward dynamics over long horizons = better planning

Zero-shot Planning

Table 1. Planning results for offline world models on six control environments.

Model	Maze SR ↑	Wall SR ↑	Reach SR ↑	PushT SR ↑	Rope CD ↓	Granular CD ↓
IRIS	0.74	0.04	0.18	0.32	1.11	0.37
DreamerV3	**1.00**	**1.00**	0.64	0.30	2.49	1.05
TD-MPC2	0.00	0.00	0.00	0.00	2.52	1.21
Ours	0.98	0.96	**0.92**	**0.90**	**0.41**	**0.26**

Data
- if not using VQA and fully self-supervised
  - stage 1 : standard shot vid datasets
  - stage 2: ego4D; hour long videos
  - stage 3: tasks specific environments

Chunk compression
- *State*:
  - TTT (recurrence)
  - Dynamic token dropping (causal autoregressive)
- *Losses*: Future prediction, Scene Flow, VQA?

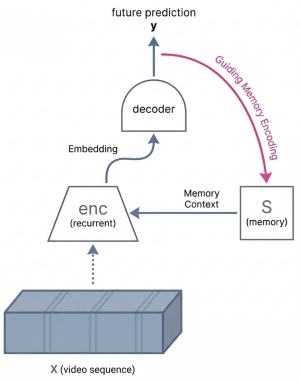
Defining a chunk
- not need if doing token dropping
- for stateful training
  - stage 1: video = chunk
  - stage 2: fixed K frames
    - can heuristics with thresholds→ covisibility/future pred
  - stage 3, task stage = chunk

Ambitious: Meta learnt

Post-training (the fun part!! 😍)
- RMA: dense supervision; privileged info = KV cache
- Quiet-Star, SEAL etc. (repurpose CL/reasoning tasks as memory management)

Evaluation
- Long horizon sims
- Ego4D
- EPM env → object retrieval/navigation

Baselines
- CroBo, ToBo
- NE-Dreamer



Stage 1: Learning Temporal Dynamics

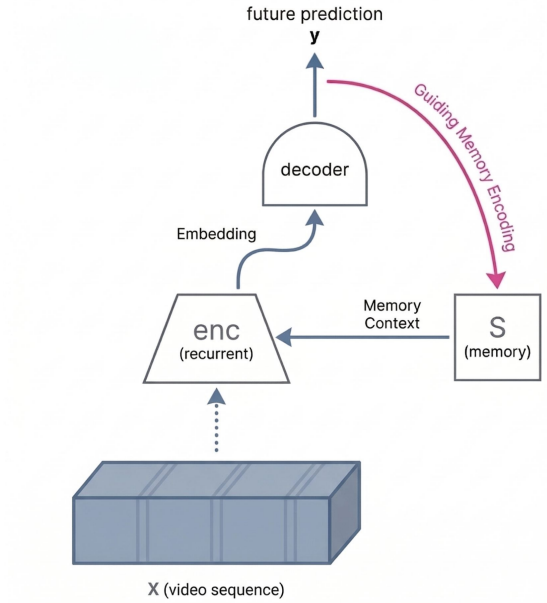
Goal: Compress video into a compact state that captures how things move, not just how they look

Architecture:

- Recurrent encoder compresses video into a compact spatio-temporal state
- Bounded memory footprint. We simply cross-attend to the latent memory

Training:

- Self-supervised: predict future DINO features
- Auxiliary loss: scene flow for more expressive state



The compressed state encodes environment dynamics: *Cars decelerate. Pedestrians change direction. Occlusions resolve*

Stage 2: Long Horizon Forecasting

From Reaction to Anticipation: *A 1km construction zone. Reactive systems brake hard at the cones. A long-horizon model sees the signs 30 seconds earlier, smoothly decelerates, and finds a merge gap*

How:

- Stage 1 encoder compresses video chunks into compact states
- Encoder-Decoder Autoregressive transformer predicts across chunks (parallelizable training)
- Long-horizon BPTT is infeasible → this staged approach makes it tractable

Encoded representation understands compositional structure of tasks: "highway merge" = check-mirror → signal → gap-find → accelerate → merge → stabilize.

Such representation would yield sample-efficient and robust policies.

Stage 3: World Modelling

The leap: State + Action → Next State. Now, the model can "imagine" consequences of steering/braking decisions before executing them.

- **Planning**: MPC / tree-search over imagined futures
- **Policy training**: Train policies inside the world model (no real-world rollouts needed)
- **Simulation at scale**: The model IS the simulator — learned from data, not engineered

The Full Stack for General-Purpose Robotics:

```
LLM:  "Merge left, take the exit in 800m"
  ↓
WM:   "Accelerate to 120, truck clears in 3s — safe gap"
  ↓
 π:   Steering + throttle at 20Hz
```

The LLM knows what to do. The world model figures out how. The policy does it.